

WYKŁAD 7

Programowanie obiektowe II: zaawansowane OOP

Kurs języka Python — semestr 2025/2026

Wprowadzenie do programowania II | Informatyka Stosowana

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- | | | |
|---|----------------------------------|---|
| 1 | Dziedziczenie wielokrotne | problem diamentu, <code>super()</code> i MRO |
| 2 | MRO — C3 linearization | algorytm, <code>.mro()</code> , niuanse |
| 3 | Mixiny | kompozycja zachowań bez hierarchii |
| 4 | ABC — klasy abstrakcyjne | <code>@abstractmethod</code> , wymuszanie interfejsu |
| 5 | Dataclasses i NamedTuple | <code>@dataclass</code> , <code>field()</code> , <code>frozen</code> , porównanie |
| 6 | Protokoły i duck typing | <code>typing.Protocol</code> , structural subtyping |
| 7 | Deskryptory | <code>__get__</code> , <code>__set__</code> , data vs non-data |
| 8 | Wzorce projektowe | Factory, Strategy, Observer, metaklasy |

Dziedziczenie wielokrotne

wiele klas bazowych, problem diamentu, `super()` i MRO

Dziedziczenie wielokrotne - składnia

Python pozwala dziedziczyć z wielu klas naraz — w odróżnieniu od Javy czy C#

```
class A:
    def metoda_a(self):
        return 'A'

class B:
    def metoda_b(self):
        return 'B'

class C(A, B):    # dziedziczy z A i B
    pass

c = C()
print(c.metoda_a()) # A
print(c.metoda_b()) # B
print(isinstance(c, A)) # True
print(isinstance(c, B)) # True
```

Kolejność ma znaczenie!

class C(A, B)

Python szuka metod najpierw
w A, potem w B

class C(B, A)

Odwrotnie — najpierw B

Kolejność klas bazowych
określa priorytet metod.

To podstawa MRO — o którym
za chwilę.

Problem diamentu (diamond problem)

Co się dzieje, gdy dwie klasy bazowe mają wspólnego przodka?

```
class Zwierze:
    def __init__(self):
        print('Zwierze.__init__')

class Latajace(Zwierze):
    def __init__(self):
        super().__init__()
        print('Latajace.__init__')

class Plywajace(Zwierze):
    def __init__(self):
        super().__init__()
        print('Plywajace.__init__')

class Kaczka(Latajace, Plywajace):
    def __init__(self):
        super().__init__()
        print('Kaczka.__init__')
```

Wynik Kaczka():

```
Zwierze.__init__
Plywajace.__init__
Latajace.__init__
Kaczka.__init__
```

Dzięki MRO i `super()`,
`Zwierze.__init__` wywołane
tylko RAZ — nie dwa razy.
To rozwiązanie diamentu.

MRO - Method Resolution Order

Algorytm C3 linearization określa kolejność przeszukiwania klas

Zasada C3:

$$L[C(B1...BN)] = C + \text{merge}(L[B1], \dots, L[BN], B1...BN)$$

Procedura merge: weź czoło pierwszej listy. Jeśli nie występuje w ogonie żadnej innej listy — dodaj do wyniku i usuń wszędzie. Jeśli występuje — spróbuj następnej listy. Powtarzaj aż wszystko scalone lub błąd.

Prosty przypadek

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
```

```
# MRO: D -> B -> C -> A -> object
```

Weryfikacja

```
print(D.mro())
# [<class 'D'>, <class 'B'>,
#  <class 'C'>, <class 'A'>,
#  <class 'object'>]

print(D.__mro__) # to samo, tuple
```

MRO - złożony przykład krok po kroku

```
class O: pass
class F(O): pass
class E(O): pass
class D(O): pass
class C(D, F): pass
class B(D, E): pass
class A(B, C): pass
```

Obliczenia:

$L[O] = O$
 $L[F] = F \ O$ $L[E] = E \ O$ $L[D] = D \ O$

$L[C] = C + \text{merge}(DO, FO, DF)$
 $= C \ D + \text{merge}(O, FO, F)$
 $= C \ D \ F + \text{merge}(O, O)$
 $= C \ D \ F \ O$

$L[B] = B + \text{merge}(DO, EO, DE)$
 $= B \ D + \text{merge}(O, EO, E)$
 $= B \ D \ E + \text{merge}(O, O)$
 $= B \ D \ E \ O$

$L[A] = A + \text{merge}(BDEO, CDFO, BC)$
 $= A \ B + \text{merge}(DEO, CDFO, C)$
 $= A \ B \ C + \text{merge}(DEO, DFO)$
 $= A \ B \ C \ D + \text{merge}(EO, FO)$
 $= A \ B \ C \ D \ E \ F \ O$

Wynik: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow O$ — klasy niższe w hierarchii mają wyższy priorytet

super() podąża za MRO

super() nie znaczy „klasa bazowa” - znaczy „następna w MRO”

```
class Baza:
    def __init__(self):
        print('Baza')
        super().__init__() # -> object

class A(Baza):
    def __init__(self):
        print('A')
        super().__init__() # -> C (nie Baza!)

class C(Baza):
    def __init__(self):
        print('C')
        super().__init__() # -> Baza

class D(A, C): # MRO: D A C Baza object
    def __init__(self):
        print('D')
        super().__init__() # -> A
```

D() wypisze:

D

A

C

Baza

⚠ super() w A nie idzie
do Baza, lecz do C!
Bo MRO dla D to:
D → A → C → Baza → object

Nieemożliwe MRO - TypeError

✗ Algorytm C3 nie daje rady

```
class Baza: pass
class B(Baza): pass
class C(B): pass
class D(B, C): pass # TypeError!
```

```
# Cannot create a consistent
# method resolution order (MRO)
# for bases B, C
```

✓ Poprawna kolejność

```
class Baza: pass
class B(Baza): pass
class C(B): pass
class D(C, B): pass # OK!
```

```
# MRO: D -> C -> B -> Baza
# C jest bardziej szczegółowa
# niż B, więc musi być pierwsza
```

Zasada: klasy bardziej szczegółowe (potomne) muszą być wymienione przed ogólniejszymi (bazowymi).

Mixiny

kompozycja zachowań bez budowania głębokiej hierarchii

Czym jest mixin?

Mała klasa dodająca jedno konkretne zachowanie - nie jest samodzielnym bytem

```
import json

class JsonMixin:
    def to_json(self):
        return json.dumps(self.__dict__)

    @classmethod
    def from_json(cls, data):
        return cls(**json.loads(data))

class ReprMixin:
    def __repr__(self):
        attrs = ', '.join(
            f'{k}={v!r}'
            for k, v in self.__dict__.items()
        )
        return f'{type(self).__name__}({attrs})'
```

Użycie:

```
class Student(JsonMixin, ReprMixin):
    def __init__(self, imie, wiek):
        self.imie = imie
        self.wiek = wiek

s = Student('Jan', 22)
print(repr(s))
# Student(imie='Jan', wiek=22)

j = s.to_json()
# '{"imie": "Jan", "wiek": 22}'

s2 = Student.from_json(j)
print(s2.imie) # Jan
```

Konwencja: mixin w nazwie (JsonMixin, LoggingMixin). Mixin nie definiuje `__init__` - dodaje metody.

Mixin vs dziedziczenie vs kompozycja

Cecha	Dziedziczenie	Mixin	Kompozycja
Relacja	is-a (jest)	acts-as (zachowuje się)	has-a (ma)
<code>__init__</code>	tak, wywołuje <code>super()</code>	nie definiuje	tworzy wewn. obiekt
Złożoność	głęboka hierarchia	płaska, modularna	płaska
Przykład	Pies(Zwierze)	Student(JsonMixin)	Auto(silnik=V8())
Kiedy?	prawdziwa relacja typow	dodanie zachowania	delegowanie pracy

Zasada: preferuj kompozycję. Używaj mixinów do dodania zachowań. Dziedziczenie tylko dla prawdziwego is-a.

ABC - klasy abstrakcyjne

wymuszanie interfejsu, @abstractmethod, @abstractproperty

Klasa abstrakcyjna - ABC + @abstractmethod

Nie można utworzyć instancji klasy abstrakcyjnej - wymusza implementację w podklasach

```
from abc import ABC, abstractmethod

class Figura(ABC):
    @abstractmethod
    def pole(self):
        """Zwraca pole figury."""
        pass

    @abstractmethod
    def obwod(self):
        pass

    def opis(self):
        # zwykła metoda - dziedziczona
        return f'{type(self).__name__}: pole={self.pole():.2f}'

# Figura() # TypeError: Can't instantiate abstract class

class Kolo(Figura):
    def __init__(self, r):
        self.r = r
    def pole(self):
        return 3.14159 * self.r ** 2
    def obwod(self):
        return 2 * 3.14159 * self.r

k = Kolo(5)
print(k.opis()) # Kolo: pole=78.54
```

ABC - zapomnienie implementacji

✗ Brak implementacji obwod()

```
class Prostokat(Figura):
    def __init__(self, a, b):
        self.a, self.b = a, b
    def pole(self):
        return self.a * self.b
    # brak obwod()!
```

```
Prostokat(3, 4)
# TypeError: Can't instantiate
# abstract class Prostokat with
# abstract method obwod
```

✓ Kompletna implementacja

```
class Prostokat(Figura):
    def __init__(self, a, b):
        self.a, self.b = a, b
    def pole(self):
        return self.a * self.b
    def obwod(self):
        return 2 * (self.a + self.b)
```

```
p = Prostokat(3, 4)
print(p.pole())    # 12
print(p.obwod())   # 14
```

ABC gwarantuje, że każda podklasa implementuje wymagany interfejs. Błąd wykapywany na etapie tworzenia obiektu.

ABC - abstract property i wywołanie super()

Abstract property

```
from abc import ABC, abstractmethod
```

```
class Pojazd(ABC):  
    @property  
    @abstractmethod  
    def predkosc_max(self):  
        pass
```

```
class Samochod(Pojazd):  
    @property  
    def predkosc_max(self):  
        return 200
```

```
s = Samochod()  
print(s.predkosc_max) # 200
```

super() w metodzie abstrakcyjnej

```
class AbstractClass(ABC):  
    @abstractmethod  
    def do_something(self):  
        print('Dzialanie ogolne!')
```

```
class Concrete(AbstractClass):  
    def do_something(self):  
        super().do_something()  
        print('Dzialanie szczegolowe!')
```

```
c = Concrete()  
c.do_something()  
# Dzialanie ogolne!  
# Dzialanie szczegolowe!
```

Metoda abstrakcyjna może mieć implementację! Podklasa i tak musi ją nadpisać, ale może wywołać super().

Dataclasses i NamedTuple

@dataclass — mniej boilerplate'u, więcej treści

@dataclass - automatyczny boilerplate

Python 3.7+ — dekorator generuje `__init__`, `__repr__`, `__eq__` automatycznie

Bez dataclass (boilerplate)

```
class Punkt:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __repr__(self):
        return f'Punkt(x={self.x}, y={self.y})'
    def __eq__(self, other):
        if not isinstance(other, Punkt):
            return NotImplemented
        return (self.x, self.y) == \
            (other.x, other.y)
```

Z @dataclass (3 linie!)

```
from dataclasses import dataclass

@dataclass
class Punkt:
    x: float
    y: float

p1 = Punkt(1.0, 2.0)
p2 = Punkt(1.0, 2.0)
print(p1)          # Punkt(x=1.0, y=2.0)
print(p1 == p2)    # True
```

@dataclass generuje: `__init__`, `__repr__`, `__eq__`. Opcjonalnie: `__hash__`, `__lt__`/`__gt__` (order=True).

@dataclass - field() i wartości domyślne

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class Student:
    imie: str
    nazwisko: str
    wiek: int = 20 # prosta wartosc domyslna
    oceny: List[float] = field(default_factory=list) # MUSI byc field()!
    _id: int = field(init=False, repr=False) # pominięte w __init__ i __repr__

    def __post_init__(self): # wywoływane PO __init__
        self._id = hash((self.imie, self.nazwisko))
        if self.wiek < 0:
            raise ValueError('Wiek nie może być ujemny')

s = Student('Jan', 'Nowak')
print(s) # Student(imie='Jan', nazwisko='Nowak', wiek=20, oceny=[])
print(s._id) # hash – ale nie widac w repr

s2 = Student('Anna', 'Kowalska', wiek=-5) # ValueError!
```

@dataclass - frozen i order

frozen=True — niemutowalny

```
@dataclass(frozen=True)
class Punkt:
    x: float
    y: float

p = Punkt(1.0, 2.0)
p.x = 5 # FrozenInstanceError!

# frozen daje tez __hash__
d = {p: 'OK'} # mozna uzyc
               # jako klucz!
```

order=True — porównywanie

```
@dataclass(order=True)
class Wersja:
    major: int
    minor: int
    patch: int = 0

v1 = Wersja(3, 9, 1)
v2 = Wersja(3, 10, 0)
print(v1 < v2) # True

wersje = [v2, v1, Wersja(2, 7)]
print(sorted(wersje))
# [Wersja(2,7,0),Wersja(3,9,1),...]
```

Opcje @dataclass:

```
@dataclass(init=True, repr=True, eq=True, order=False, frozen=False,
           unsafe_hash=False, match_args=True, kw_only=False, slots=False)

# Python 3.10+: slots=True automatycznie generuje __slots__!
# Python 3.10+: kw_only=True wymusza argumenty nazwane w __init__
```

@dataclass vs NamedTuple vs zwykła klasa

Cecha	@dataclass	NamedTuple	Zwykła klasa
Mutowalny?	domyślnie tak	NIE (tuple!)	tak
__init__	auto	auto	ręcznie
__repr__	auto	auto	ręcznie
__eq__	auto (wartości)	auto (wartości)	domyślnie is
__hash__	frozen=True	tak (niezmienny)	ręcznie
Dziedziczenie	pełne	ograniczone	pełne
Slicing/index	nie	tak (to tuple!)	nie
Kiedy?	typowe obiekty	lekkie rekordy	złożona logika

NamedTuple - lekkie rekordy

Stary styl (collections)

```
from collections import namedtuple

Punkt = namedtuple('Punkt', ['x', 'y'])
p = Punkt(1, 2)
print(p.x, p.y)      # 1 2
print(p[0], p[1])    # 1 2 (tuple!)

x, y = p # rozpakowywanie
print(p._asdict())   # {'x': 1, 'y': 2}
```

Nowy styl (typing) — zalecany

```
from typing import NamedTuple

class Punkt(NamedTuple):
    x: float
    y: float
    kolor: str = 'red' # domyslne

p = Punkt(1.0, 2.0)
print(p) # Punkt(x=1.0, y=2.0, kolor='red')

# Niemutowalny!
p.x = 5 # AttributeError
```

NamedTuple to tuple z nazwanymi polami. Lekki, szybki, niemutowalny. Idealny do prostych rekordów (punkt, kolor, config).

Protokoły i duck typing

jeśli chodzi jak kaczka i kwacze jak kaczka...

Duck typing - filozofia Pythona

"If it walks like a duck and quacks like a duck, it must be a duck"

Przykład — brak interfejsu!

```
class Plik:
    def read(self):
        return 'dane z pliku'

class API:
    def read(self):
        return 'dane z API'

class Baza:
    def read(self):
        return 'dane z bazy'

def przetworz(zrodlo):
    dane = zrodlo.read() # duck typing!
    return dane.upper()

print(przetworz(Plik())) # DANE Z PLIKU
print(przetworz(API())) # DANE Z API
print(przetworz(Baza())) # DANE Z BAZY
```

Kluczowe cechy

Nie sprawdzamy typu

— sprawdzamy zachowanie

Żadna z klas nie dziedziczy

ze wspólnej klasy bazowej

EAFP > LBYL:

Easier to Ask Forgiveness
than Permission

To fundament Pythona:

elastyczność > formalizm

typing.Protocol - structural subtyping

Python 3.8+ - formalny duck typing z podpowiedziami typów

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Readable(Protocol):
    def read(self) -> str: ...    # tylko sygnatura

class Plik:
    def read(self) -> str:
        return 'dane z pliku'

class Liczba:
    def __init__(self, n): self.n = n

# Plik NIE dziedziczy z Readable, ale pasuje strukturalnie!
print(isinstance(Plik(), Readable))    # True
print(isinstance(Liczba(5), Readable)) # False

def przetworz(zrodlo: Readable) -> str: # type hint!
    return zrodlo.read().upper()

print(przetworz(Plik()))    # DANE Z PLIKU
# przetworz(Liczba(5))      # mpy: error! Liczba nie ma .read()
```

__init_subclass__ - hook rejestracyjny

Python 3.6+ — wywoływane automatycznie gdy klasa zostanie podklasowana

```
class Plugin:
    _registry = []

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        cls._registry.append(cls)
        print(f'Zarejestrowano: {cls.__name__}')

class PDF(Plugin): pass          # Zarejestrowano: PDF
class CSV(Plugin): pass         # Zarejestrowano: CSV
class Excel(Plugin): pass       # Zarejestrowano: Excel

print(Plugin._registry)
# [<class 'PDF'>, <class 'CSV'>, <class 'Excel'>]

# Wzorzec registry bez metaklas!
```

Lekka alternatywa dla metaklas — automatyczna rejestracja pluginów, walidacja podklas, wymuszanie konwencji.

Enum - wyliczenia jako klasy

```
from enum import Enum, auto

class Kolor(Enum):
    CZERWONY = 1
    ZIELONY = 2
    NIEBIESKI = 3

class Status(Enum):
    AKTYWNY = auto() # auto = 1
    ZAWIESZONY = auto() # auto = 2
    USUNIETY = auto() # auto = 3

print(Kolor.CZERWONY) # Kolor.CZERWONY
print(Kolor.CZERWONY.value) # 1
print(Kolor.CZERWONY.name) # CZERWONY
print(Kolor(2)) # Kolor.ZIELONY
```

Użycie w praktyce

```
# Iteracja
for k in Kolor:
    print(k.name, k.value)

# Porównanie
if status == Status.AKTYWNY:
    print('Aktywny!')

# IntEnum - porównywalne z int
from enum import IntEnum
class Prio(IntEnum):
    LOW = 1
    MED = 2
    HIGH = 3

print(Prio.HIGH > 2) # True
print(Prio.HIGH + 1) # 4
```

Enum zamiast magicznych stałych: czytelniejszy kod, lepsza walidacja, łatwiejsze debugowanie.

Deskryptory

`__get__`, `__set__`, `__delete__` — mechanizm pod maską `@property`

Deskryptory - `__get__`, `__set__`, `__delete__`

Obiekt, który kontroluje dostęp do atrybutu innego obiektu

```
class Walidator:
    def __set_name__(self, owner, name):
        self.name = name # nazwa atrybutu
    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if not isinstance(value, int) or value < 0:
            raise ValueError(f'{self.name} musi byc int >= 0, got {value}')
        obj.__dict__[self.name] = value

class Produkt:
    cena = Walidator() # deskryptor!
    ilosc = Walidator()

p = Produkt()
p.cena = 100 # OK
p.cena = -10 # ValueError: cena musi byc int >= 0
```

@property to właśnie deskryptor! Python pod spodem używa `__get__`/`__set__` z klasy property.

Data vs non-data descriptor

Data descriptor

Ma `__get__` + `__set__`

Priorytet NAD `__dict__` obiektu!

Przykład: `@property`

`obj.attr -> deskryptor.__get__`
`obj.attr = v -> deskryptor.__set__`
(nawet jeśli `attr` jest w `obj.__dict__`)

Non-data descriptor

Ma tylko `__get__`

`obj.__dict__` ma priorytet!

Przykład: zwykłe metody

(metody to też deskryptory —
funkcja w klasie ma `__get__`
który binduje `self`)

Priorytet: data descriptor > `obj.__dict__` > non-data descriptor > `klasa.__dict__` > `AttributeError`

Wzorce projektowe

Factory, Strategy, Observer — OOP w praktyce

Factory Method - fabryka obiektów

```
from abc import ABC, abstractmethod

class Pojazd(ABC):
    @abstractmethod
    def jedz(self): pass

class Samochod(Pojazd):
    def jedz(self): return 'Jade samochodem!'

class Rower(Pojazd):
    def jedz(self): return 'Jade rowerem!'

class FabrykaPojazdów:
    _registry = {'samochod': Samochod, 'rower': Rower}

    @classmethod
    def stworz(cls, typ):
        klasa = cls._registry.get(typ)
        if klasa is None:
            raise ValueError(f'Nieznany typ: {typ}')
        return klasa()

    @classmethod
    def zarejestruj(cls, typ, klasa):
        cls._registry[typ] = klasa
```

```
auto = FabrykaPojazdów.stworz('samochod')
print(auto.jedz()) # Jade samochodem!
```


Strategy - wymienna strategia

```
from abc import ABC, abstractmethod

class Sortowanie(ABC):
    @abstractmethod
    def sortuj(self, dane): pass

class BubbleSort(Sortowanie):
    def sortuj(self, dane):
        d = dane[:]
        for i in range(len(d)):
            for j in range(len(d)-1-i):
                if d[j] > d[j+1]: d[j], d[j+1] = d[j+1], d[j]
        return d

class BuiltinSort(Sortowanie):
    def sortuj(self, dane): return sorted(dane)

class Sorter:
    def __init__(self, strategia: Sortowanie):
        self.strategia = strategia

    def wykonaj(self, dane):
        return self.strategia.sortuj(dane)

s = Sorter(BubbleSort())
print(s.wykonaj([3,1,2])) # [1, 2, 3]
s.strategia = BuiltinSort() # zmiana w runtime!
print(s.wykonaj([3,1,2])) # [1, 2, 3]
```

Observer - powiadamianie o zmianach

```
class Wydarzenie:
    def __init__(self):
        self._obserwatorzy = []
    def subskrybuj(self, fn):
        self._obserwatorzy.append(fn)
    def powiadom(self, *args, **kwargs):
        for fn in self._obserwatorzy:
            fn(*args, **kwargs)

class Sklep:
    def __init__(self):
        self.nowy_produkt = Wydarzenie()
    def dodaj_produkt(self, nazwa):
        print(f'Dodano: {nazwa}')
        self.nowy_produkt.powiadom(nazwa)

def email_alert(nazwa):
    print(f' Email: nowy produkt {nazwa}!')
def sms_alert(nazwa):
    print(f' SMS: nowy produkt {nazwa}!')

sklep = Sklep()
sklep.nowy_produkt.subskrybuj(email_alert)
sklep.nowy_produkt.subskrybuj(sms_alert)
sklep.dodaj_produkt('Laptop') # Dodano: Laptop / Email: ... / SMS: ...
```

Context manager jako klasa

`__enter__` i `__exit__` — automatyczne zarządzanie zasobami

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.time()
        return self

    def __exit__(self, *args):
        import time
        self.elapsed = time.time() - self.start
        print(f'Czas: {self.elapsed:.3f}s')
        return False # nie tłumimy wyjatkov

with Timer() as t:
    sum(range(10*7))
# Czas: 0.234s
```

```
def __exit__(self, exc_type, exc_val, exc_tb):
    # typ wyjatku
    # wartosc wyjatku
    # traceback
    if exc_type is not None:
        print(f'Bład: {exc_val}')
    return False
    # True = stlum wyjatek
    # False = propaguj dalej

# Odpowiednik contextlib:
from contextlib import contextmanager

@contextmanager
def timer():
    import time
    start = time.time()
    yield # tu wykonuje sie blok with
    print(f'Czas: {time.time() - start:.3f}s')
```

Metaklasy - fabryki klas

Każda klasa jest instancją metaklasy `type` — `type(type)` to `type`!

Dynamiczne tworzenie klasy

```
# type(name, bases, dict)
Foo = type('Foo', (), {'x': 42})

f = Foo()
print(f.x)          # 42
print(type(Foo))    # <class 'type'>
print(type(type))   # <class 'type'>
```

Własna metaklasa

```
class Meta(type):
    def __init__(cls, name, bases, dct):
        cls.created = True
        super().__init__(name, bases, dct)

class Foo(metaclass=Meta):
    pass

print(Foo.created)  # True
```

⚠ Metaklasy to potężne narzędzie, ale w 99% przypadków wystarczy:
`@dataclass`, `__init_subclass__`, dekoratory klas. Metaklasy = ostateczność.

Pełny obraz tworzenia obiektu

```
# Pełny lancuch:
# Foo() -> type.__call__(Foo) -> Foo.__new__(Foo) -> Foo.__init__(self)

# 1. Metaklasa type wywołuje __call__
# 2. __call__ wywołuje Foo.__new__ (alokacja)
# 3. Jeśli __new__ zwróci instancję Foo -> wywołuje __init__

class Verbose(type):
    def __call__(cls, *args, **kwargs):
        print(f'1. type.__call__({cls.__name__})')
        obj = cls.__new__(cls, *args, **kwargs)
        print(f'2. __new__ zwrócił: {type(obj).__name__}')
        if isinstance(obj, cls):
            obj.__init__(*args, **kwargs)
            print(f'3. __init__ wykonany')
        return obj

class Foo(metaclass=Verbose):
    def __new__(cls, x):
        print(f' Foo.__new__(x={x})')
        return super().__new__(cls)
    def __init__(self, x):
        print(f' Foo.__init__(x={x})')
        self.x = x

f = Foo(42)
# 1. type.__call__(Foo)
# Foo.__new__(x=42)
# 2. __new__ zwrócił: Foo
# Foo.__init__(x=42)
# 3. __init__ wykonany
```

Fabryka abstrakcyjna - przykład z labów

```
from abc import ABC, abstractmethod

class Pojazd(ABC):
    def __init__(self, model, rok):
        self._model, self._rok = model, rok
        self._predkosc = 0
    @property
    def predkosc(self): return self._predkosc
    @predkosc.setter
    def predkosc(self, v):
        if v < 0: raise ValueError('Predkosc nie moze byc ujemna')
        self._predkosc = v
    @predkosc.deleter
    def predkosc(self): self._predkosc = 0

class Samochod(Pojazd):
    def __init__(self, model, rok, drzwi=4):
        super().__init__(model, rok)
        self.drzwi = drzwi

class FabrykaPojazdow(ABC):
    _licznik = 0
    @abstractmethod
    def stworz_pojazd(self, model, rok): pass
    @classmethod
    def utworz_fabryke(cls, typ):
        fabryki = {'samochod': FabrykaSamochodow, 'autobus': FabrykaAutobusow}
        return fabryki[typ]()
    @staticmethod
    def sprawdz_rok(rok): return 1900 <= rok <= 2025
```

singledispatchmethod - polimorfizm w metodach

```
from functools import singledispatchmethod

class EventHandler:
    @singledispatchmethod
    def handle(self, event):
        print(f'Nieznany typ: {type(event)}')

    @handle.register(str)
    def _(self, event):
        print(f'String event: {event}')

    @handle.register(int)
    def _(self, event):
        print(f'Int event: kod {event}')

    @handle.register(dict)
    def _(self, event):
        print(f'Dict event: {event.keys()}')

h = EventHandler()
h.handle('klik')          # String event: klik
h.handle(404)             # Int event: kod 404
h.handle({'a': 1})        # Dict event: dict_keys(['a'])
h.handle([1, 2, 3])       # Nieznany typ: <class 'list'>
```

__slots__ - wydajność z dziedziczeniem

__slots__ w hierarchii

```
class Baza:
    __slots__ = ('x', 'y')

class Potomna(Baza):
    __slots__ = ('z',) # TYLKO nowe!
    # x, y dziedziczone z Baza

p = Potomna()
p.x = 1; p.y = 2; p.z = 3 # OK
p.w = 4 # AttributeError

# Uwaga: jeśli Potomna NIE definiuje
# __slots__, dostaje __dict__!
```

Benchmark: 1M obiektów

```
import sys

class Normal:
    def __init__(self, x, y):
        self.x, self.y = x, y

class Slotted:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x, self.y = x, y

n = Normal(1, 2)
s = Slotted(1, 2)
print(sys.getsizeof(n)) # ~152 bytes
print(sys.getsizeof(s)) # ~56 bytes
# ~3x mniej pamięci!
```

Python 3.10+: @dataclass(slots=True) generuje __slots__ automatycznie!

Dekorator klasy - alternatywa dla metaklas

```
def add_greeting(cls):
    """Dekorator dodajacy metode do klasy."""
    def powitaj(self):
        return f'Czesc, jestem {self.imie}!'
    cls.powitaj = powitaj
    return cls

def singleton(cls):
    """Dekorator implementujacy Singleton."""
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@add_greeting
class Student:
    def __init__(self, imie): self.imie = imie

@singleton
class Database:
    def __init__(self): print('Polaczenie z baza!')

s = Student('Jan')
print(s.powitaj())    # Czesc, jestem Jan!

db1 = Database()      # Polaczenie z baza!
db2 = Database()      # (cisza - ten sam obiekt)
print(db1 is db2)     # True
```

Dobre praktyki OOP II

Kompozycja > dziedziczenie

Używaj has-a zamiast is-a gdy to możliwe

Mixiny z umiarem

Jedna klasa, jedno zachowanie. Nie 15 mixinów naraz.

ABC dla interfejsów

Wymuszaj kontrakt gdy klasa bazowa nie ma sensu sama w sobie

@dataclass dla danych

Nie pisz ręcznie `__init__`/`__repr__`/`__eq__` gdy to zbyteczne

typing.Protocol > ABC

Gdy nie potrzebujesz wymuszania, tylko dokumentacji

`__init_subclass__` > metaklasy

Prostszy hook rejestracyjny w 90% przypadków

Nie optymalizuj za wcześnie

`__slots__` i frozen tylko gdy potrzebna wydajność

SOLID i DRY

Single Responsibility, Open/Closed, Don't Repeat Yourself

Co wypisze ten kod?

```
from dataclasses import dataclass
```

```
@dataclass
class A:
    x: int = 10

class B(A):
    pass

@dataclass
class C(A):
    y: int = 20

a = A()
b = B(5)
c = C(1, 2)

print(a)           # ???
print(b)           # ???
print(c)           # ???
print(a == A(10)) # ???
print(type(b))     # ???
```

Odpowiedź:

A(x=10)

A(x=5)

C(x=1, y=2)

True

<class '__main__.B'>

B dziedziczy __init__ z A

(dataclass A generuje __init__
przyjmujący x).

C ma własny @dataclass, więc
generuje nowy __init__(x, y).

a == A(10) to True bo @dataclass
generuje __eq__ porównujący wartości.

Jaki będzie MRO?

```
class X: pass
class Y: pass
class A(X, Y): pass
class B(Y, X): pass
class C(A, B): pass

# Jakie jest MRO dla C?
# Czy ten kod w ogole zadziala?
```

Odpowiedź:

TypeError!
A wymaga X przed Y.
B wymaga Y przed X.
Sprzeczność — C3 nie da
radę zbudować spójnego MRO.

To klasyczny przykład konfliktu
w wielokrotnym dziedziczeniu.

Taki konflikt wskazuje na problem w architekturze — wymaga restrukturyzacji hierarchii klas.

Kiedy czego używać?

Potrzebujesz prostego kontenera danych?

→ `@dataclass` lub `NamedTuple`

Dane są niemutowalne i lekkie?

→ `NamedTuple` lub `@dataclass(frozen=True)`

Chcesz wymusić interfejs w podklasach?

→ `ABC` + `@abstractmethod`

Chcesz dodać zachowanie bez hierarchii?

→ `Mixin`

Potrzebujesz walidacji atrybutów?

→ `@property` lub `deskryptor`

Chcesz zarejestrować podklasy?

→ `__init_subclass__`

Chcesz polimorfizmu bez ABC?

→ `typing.Protocol`

Stałe, enumeracje?

→ `Enum` / `IntEnum`

Zaawansowana kontrola tworzenia klas?

→ `Metaklasa` (ostateczność!)

Programowanie obiektowe II

1 Dziedziczenie wielokrotne

kolejność klas bazowych ma znaczenie

2 MRO (C3 linearization)

`.mro()` — `super()` podąża za MRO, nie za hierarchią

3 Mixiny

małe klasy dodające zachowanie, bez `__init__`

4 ABC + `@abstractmethod`

wymuszanie interfejsu, błąd przy tworzeniu instancji

5 `@dataclass`

automatyczny `__init__`, `__repr__`, `__eq__`, `frozen`, `order`

6 `NamedTuple`

lekki niemutowalny rekord, dostęp po nazwie i indeksie

7 `typing.Protocol`

structural subtyping — formalny duck typing

8 Deskryptory

`__get__`/`__set__` — mechanizm pod maską `@property`

9 Wzorce: Factory, Strategy, Observer

OOP w praktyce — elastyczna architektura

10 Metaklasy i `__init_subclass__`

zaawansowana kontrola tworzenia klas

Następny wykład: Obsługa błędów, pliki i kontekst - `try/except`, hierarchia wyjątków, `with`, `pathlib`, `JSON`, `logging`